OPC UA FileTransfer (AS 6.0)

Exported from Confluence on 2024 March 18

We reserve the right to change the content of this document without prior notice. The information contained herein is believed to be accurate as of the date of export, however, B&R makes no warranty, expressed or implied, with regards to the information contained within this document. B&R shall not be liable in the event if incidental or consequential damages in connection with or arising from the use of this information. The software names, hardware names and trademarks used in this document are registered by the respective companies.

BnR Generic pdf Template Version 2

Table of Contents

Versions ... **3**

1 Overview ... **3**

1.1 Information model ... 3

1.2 Typical use cases ... 5

1.2.1 Call methods sequentially ... 5

1.2.2 Call methods sequentially by program ... 6

1.2.3 Call file transfer functions with UaExpert ... 6

2 Demo Application ... **6**

2.1 Physical View ... 7

2.2 Configuration View ... 7

2.3 Logical View ... 7

3 Library UaFiles ... **8**

3.1 UaFiles.fun ... 8

3.2 UaFiles.typ ... 8

3.3 UaFiles.st ... 8

4 Program FTTask ... **8**

4.1 Variables.var ... 9

4.2 Main.st ... 9

4.3 UaFile.uam ... 9

5 Program FTClient ... **9**

5.1 Main.st ... 9

6 Default View ... **10**

6.1 Namespace URI ... 10

6.2 ImplementationOf references ... 10

6.3 ValueAttributeOf references ... 10

Content of this article:

Versions

Version	Date	Comment	Edited by
1.0	Oct 16, 2019	First Edition	Karl Mayr
1.1	Oct 17, 2019	Use cases added	Karl Mayr
1.2	Oct 31, 2019	Adopted to UaFiles library	Karl Mayr
1.3	Jan 08, 2020	Adopted to newest AS version	Karl Mayr
1.4	Jan 19, 2024	Update to AS 6	Philipp Winkler

1 Overview

According to the OPC UA specification Part 5, Annex C File Transfer, B&R plans to implement the corresponding object types and methods in the B&R OPC UA system.

It is not yet known when this implementation will be available. Until then, however, the basic functions can also be solved in the application, e.g. for EUROMAP applications for the transfer of a ProductionDataSet.

In the following the demo application “OpcUa_FileTransfer” is described, in which the essential basic functions of the OPC UA FileTransfer are implemented:

- Open() - Open a file on the FileType Object
- Close() - Close a file on the FileType Object
- Read() - Read the data content of the opened file in the FileType Object
- Write() - Write the data content to the opened file in the FileType Object
- GetPosition() –Get the filepointer of the opened file in the FileType Object
- SetPosition() - Set the filepointer of the opened file in the FileType Object
- CreateFile() – Creates a file on the FileDirectoryType Object

Currently not implemented are

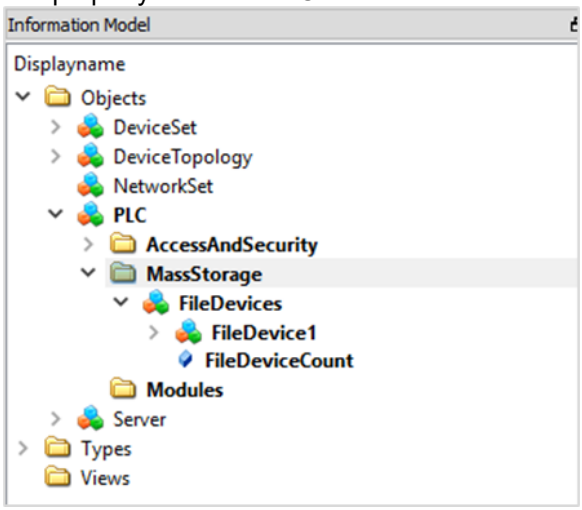
- CreateDirectory() – Creates a directory on the FileDirectoryType Object
- MoveOrCopy() – Moves or copies a file on the FileDirectoryType Object
- Delete() – Deletes a file on the FileDirectoryType Object

This demo application shows how to create, open, read, write and close one or more files on a single directory.

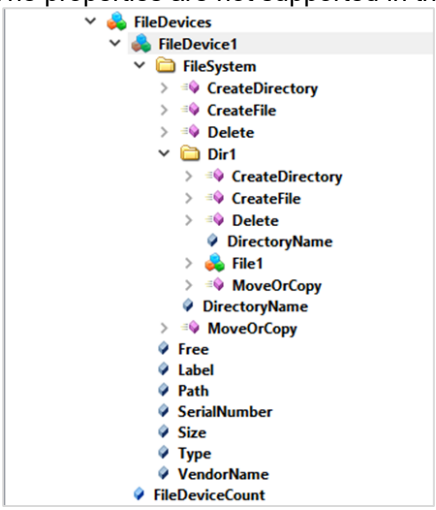
1.1 Information model

At the Automation Runtime the objects required for file transfer are managed based on the <PLC> Object in a <MassStorage> folder.
This folder contains the <FileDevices> object and in this demo the <FileDevice1> object, because we only have one device of <FileDeviceType>.

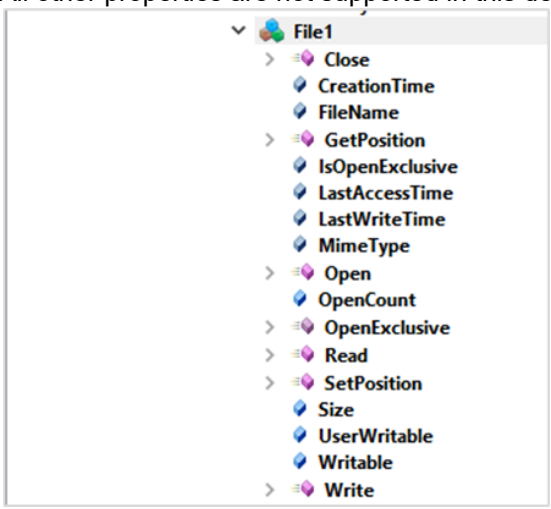
The property <FileDeviceCount> is therefore always 1.



The <FileDevice1> manages the <FileSystem> folder that contains the methods and objects for the <FileDirectoryType> derived from B&R.
It contains the folders <Dir1> and <File1> in which the files are stored.
The properties are not supported in this demo.



The <File1> contains the methods and properties to manage one file for the <FileType> derived from B&R.
The <FileName> contains the name of the file that is currently being handled and has to be supplied in this demo before <Open> this file.
All other properties are not supported in this demo.

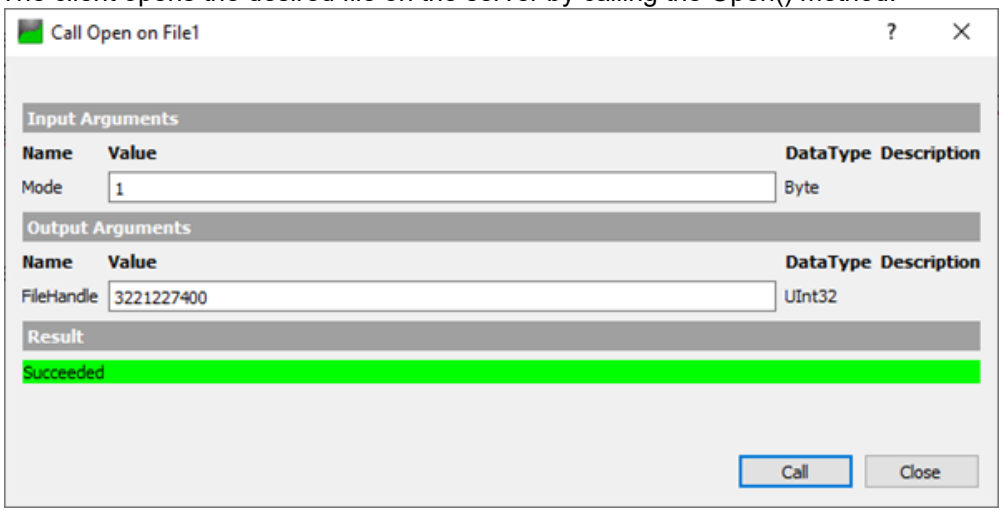


1.2 Typical use cases

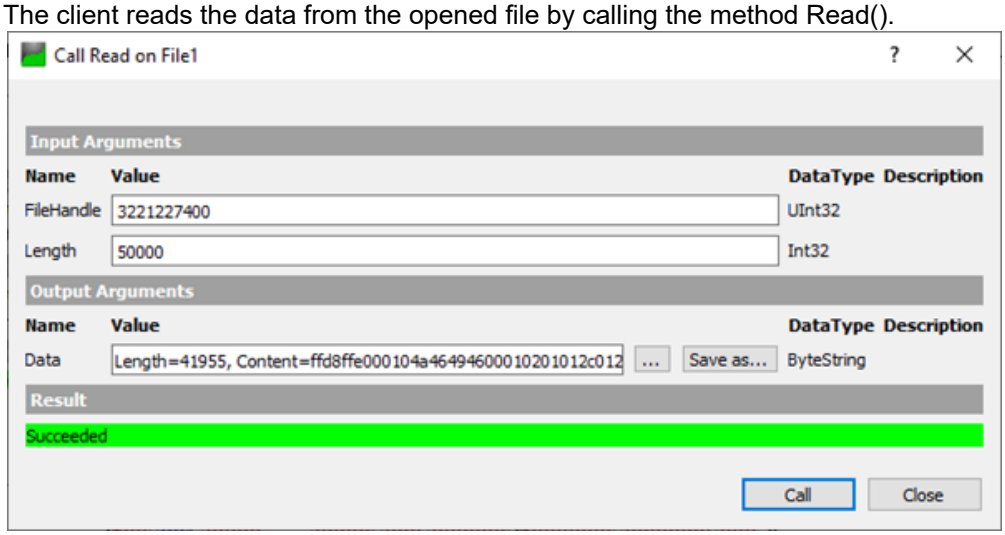
1.2.1 Call methods sequentially

A typical use case for file transfer is to copy a file from an OPC UA server to an OPC UA client is to call the methods sequentially.

- The client opens the desired file on the server by calling the Open() method.

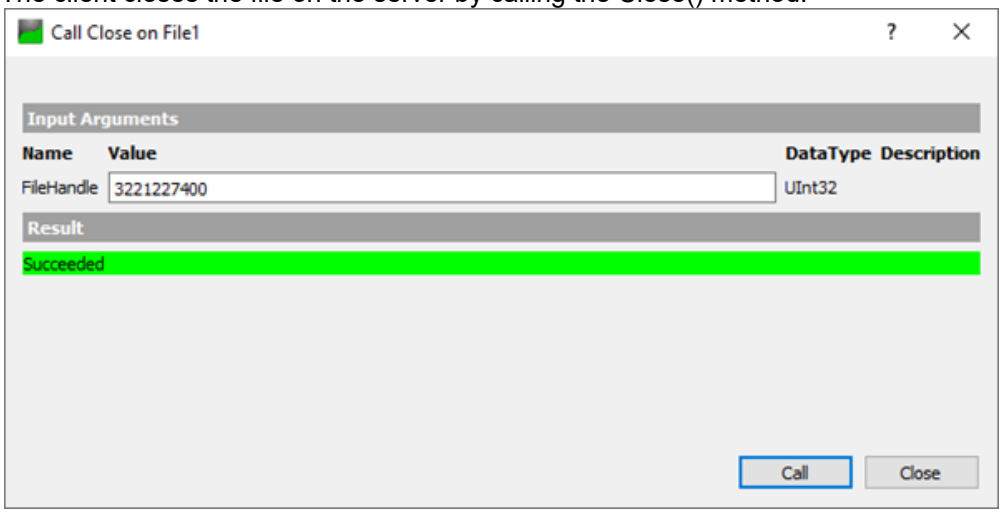


  - Mode 1 specifies that the file is to be opened read-only.
  - The OPC UA Server returns a FileHandle (in this example 3221227400), which is specified for the following calls.
  - Note: In this demo application you have to set the property File1.Filename to the filename you want to open. This filename must exist in the Path: OpcUa_FileTransfer\Temp\Simulation\FSysSim\PC\USER
- The client reads the data from the opened file by calling the method Read().



  - The FileHandle received at the Open() is passed in the FileHandle.
  - Length indicates the maximum number of bytes to be read.
  - The actual number of bytes read and the data are returned as ByteString.
  - Note: This demo application defines a read length of maximum of 50000 bytes. If the file size is more than 50000 bytes you have to call the read multiple times.

- The client closes the file on the server by calling the Close() method.



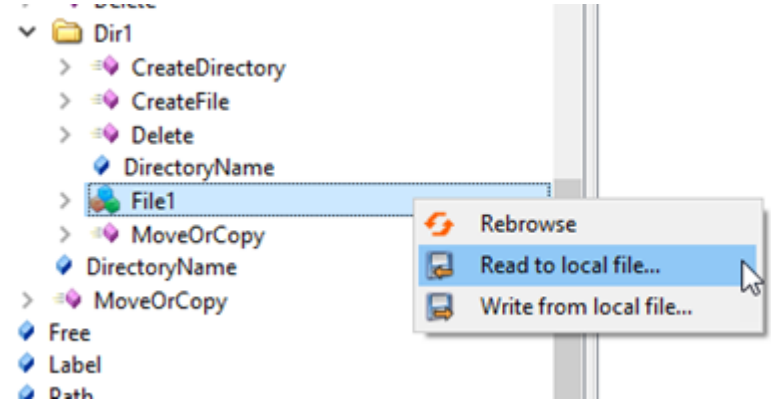
1.2.2 Call methods sequentially by program

You can call sequentially these methods also by program with our AsOpcUac library. This is shown in the demo program FTClient (see 4.3).

1.2.3 Call file transfer functions with UaExpert

Another typical use case for file transfer is copying a file from an OPC UA server to an OPC UA client with combined functions such as those available in UaExpert.

If an object of type <FileType> is selected with the right mouse button, these functions can be called:



2 Demo Application

The demo application “OpcUa_FileTransfer” is based on

- Automation Studio AS 6.0.0 and
- Automation Runtime AR A6.00

Please use this or higher versions.

This demo is installed in the Samples directory during Automation Studio installation.

E.g. C:\BrAutomation\AS48\Samples\OpcUa_FileTransfer.zip

The demo was developed on ArSim to show the functionality without the need of any hardware.

Note: The program code has not been created with the intention of achieving the highest performance. The focus is on step-by-step learning and simplicity for understanding the functions.

2.1 Physical View

Since files are transferred, a configuration in the <File devices> is necessary.
In this demo the default Simulation USER_PATH directory (OpcUa_FileTransfer\Temp\Simulation\FSysSim\PC\USER) is used.

File devices	
File device 1	
Name	UaFiles
Path	USER_PATH

Note: To be able to use the demo application, the file from File1.Filename must be stored in this path.

2.2 Configuration View

The OPC UA C/S technology package has to be switched on.

OPC UA C/S	6.0.0	6.0.0

An additional namespace (http://br-automation.com/OpcUa/FileTransfer/) must be inserted.

Additional Namespace ...			
5	http://br-automation.com/OpcUa/FileTransfer/		Imported OPC UA information model
6			*Imported OPC UA information model*

In the Configuration View the .uanodeset files for the information models are necessary.
These files contain all types and instances for file the system.

- Connectivity
  - OpcUaCs
    - UaCsConfig.uacfg
    - UaDvConfig.uadcfg
    - UaFiles.uad
    - filedevices.uanodeset
    - filetransfer.uanodeset

2.3 Logical View

The Documentation Folder contains this document.

For the file transfer the libraries for OPC UA, FileIO and AsBrStr are required.

The File Transfer functions are encapsulated in the <UaFiles> library so that they can possibly be reused in another programs.

The FTTask shows how to call the <UaFiles> library

With the program <FTClient> the OPC UA methods for the file transfer can be tested (called).

OpcUa_FileTransfer	
Documentation	
OpcUa_FileTransfer_A...	
Libraries	Global libraries
operator	This library contains function interfaces for IEC 61131-3
runtime	This library contains runtime functions for IEC tasks.
astime	The AsTime Library supports DATE_AND_TIME and TI
AslecCon	This library contains function interfaces for IEC 61131-3
AsOpcUac	The AsOpcUac library supports communication with OP
AsOpcUas	The AsOpcUas library provides interfaces to the OPC-U
standard	This library contains standard function blocks and functi
sys_lib	The SYS_LIB library contains functions for memory man
FileIO	The FileIO library provides function blocks for file handlir
AsBrStr	The AsBrStr Library contains FBKs for memory and char
FileSystem	
UaFiles	OPC UA FileSystem library
UaFiles.typ	OPC UA FileSystem data types
UaFiles.fun	OPC UA FileSystem FBK interface
UaFiles.st	OPC UA FileSystem implementation
FileTransfer	
FTTask	
Main.st	Init, cyclic, exit code
Variables.var	Local variables
UaFiles.uam	OPC UA FileSystem methods
FTClient	
Main.st	Init, cyclic, exit code
Types.typ	Local data types
Variables.var	Local variables

3 Library UaFiles

The library <UaFiles> contains the function block <UaFileSystem> and consists of

- the function block Interface <UaFiles.fun>,
- the required data types < UaFiles.typ>,
- the program code for the OPC UA methods <UaFiles.st>.

3.1 UaFiles.fun

Contains the Function Block declaration for <UaFileSystem>

Name	Type	& Reference	Scope	Constant	Red...	Value	Descri
UaFileSystem		☐					file sy
Execute	BOOL	☐	VAR_INPUT				
BrFileDevices	UaBrFileDevices...	☑	VAR_INPUT				
Status	DINT		VAR_OUTPUT				
Step	INT	☐	VAR	☐			
NamespaceIndex	UINT	☐	VAR	☐			
NamespaceIndexs	STRING[8]	☐	VAR	☐			
UaSrv_GetNamespaceIndex_0	UaSrv_GetName...	☐	VAR	☐			
ErrorID	DWORD	☐	VAR	☐			
FileOpen_0	FileOpen	☐	VAR	☐			
FileClose_0	FileClose	☐	VAR	☐			
FileRead_0	FileReadEx	☐	VAR	☐			
FileWrite_0	FileWriteEx	☐	VAR	☐			
FileCreate_0	FileCreate	☐	VAR	☐			
FileDelete_0	FileDelete	☐	VAR	☐			
fiMode	USINT	☐	VAR	☐			
fiOffset	UDINT	☐	VAR	☐			
NamespaceUri	STRING[80]	☐	VAR	☑		'http://br-automation.com/OpcUa/FileTransfer/'	

3.2 UaFiles.typ

Contains all data types according to the OPC UA specification part 5 and the derived data types from B&R.

The file transfer is handled with ByteStrings. Since the regular ByteString only has a length of 1024 bytes, a <UAByteStringFile> with a length of 50000 bytes is declared. Thus files up to a size of 50000 bytes can be transferred with the demo with one read. If the file size is more than 50000 bytes the read has to be called multiple times until the whole file data is transferred.

3.3 UaFiles.st

Contains the program code for the OPC UA methods.

- Case 1
  - It is necessary to determine the namespace index for namespace of the file transfer objects in 'http://br-automation.com/OpcUa/FileTransfer/'. If no errors occur it is stored in <NamspaceIndex>
- Case 2
  - For each method of the file transfer, a state machine is implemented.
  - Case 0 and 10:
    - The FBK UaSrv_MethodOperate() with the flag <UaMoa_CheckIsCalled> is used to query whether the corresponding method was called by a client. If yes, it is continued with case 20.
  - Case 20:
    - This case executes the function of the method. The corresponding call to the FileIO library of the Automation Runtime is passed on or mapped to it.
  - Case 30 and 40:
    - The FBK UaSrv_MethodOperate() with the flag <UaMoa_Finished> signals to the OPC UA server that the method has been processed, the output arguments are in the PLC variables and the result can be transferred to the OPC UA client.

4 Program FTTask

The program <FTTask> with its files contains the declaration as well as the initialization and the call of the function block <UaFileTransfer>.

4.1 Variables.var

Declaration of

<FileSystem> variable of type <UaFileSystem> for the OPC UA instance <FileSystem>

<FileDevices> variable of type <UaBrFileDevicesType> for the OPC UA instance <FileDevice1>

4.2 Main.st

Initialization of the structures <FileSystem> and <FileDevices>

Calls the function block <UaFileSystem> cyclically.

4.3 UaFile.uam

Contains the declaration for the OPC UA methods according the OPC UA specification part 5 and the binding to the corresponding PLC variables.

FTTask::UaFiles.uam [OPC UA Method Declaration] | UaFiles.uad [OPC UA Default View]

Name	Direction	Variable	UA DataType
File1_Open			
mode	Input	FileDevices.FileDevice1.FileS...	Byte
fileHandle	Output	FileDevices.FileDevice1.FileS...	UInt32
File1_Close			
fileHandle	Input	FileDevices.FileDevice1.FileS...	UInt32
File1_Read			
fileHandle	Input	FileDevices.FileDevice1.FileS...	UInt32
length	Input	FileDevices.FileDevice1.FileS...	Int32
data	Output	FileDevices.FileDevice1.FileS...	BrUaByteStringFile
File1_Write			
fileHandle	Input	FileDevices.FileDevice1.FileS...	UInt32
data	Input	FileDevices.FileDevice1.FileS...	BrUaByteStringFile
File1_GetPosition			
fileHandle	Input	FileDevices.FileDevice1.FileS...	UInt32
position	Output	FileDevices.FileDevice1.FileS...	UInt32
File1_SetPosition			
fileHandle	Input	FileDevices.FileDevice1.FileS...	UInt32
position	Input	FileDevices.FileDevice1.FileS...	UInt32
Dir1_CreateFile			
fileName	Input	FileDevices.FileDevice1.FileS...	String
requestFileOpen	Input	FileDevices.FileDevice1.FileS...	Boolean
fileNodeId	Output	FileDevices.FileDevice1.FileS...	UANodeID
fileHandle	Output	FileDevices.FileDevice1.FileS...	UInt32

5 Program FTClient

With the program <FTClient> the OPC UA methods for the file transfer can be tested (called).

5.1 Main.st

In CASE OpcConnect.Step it does

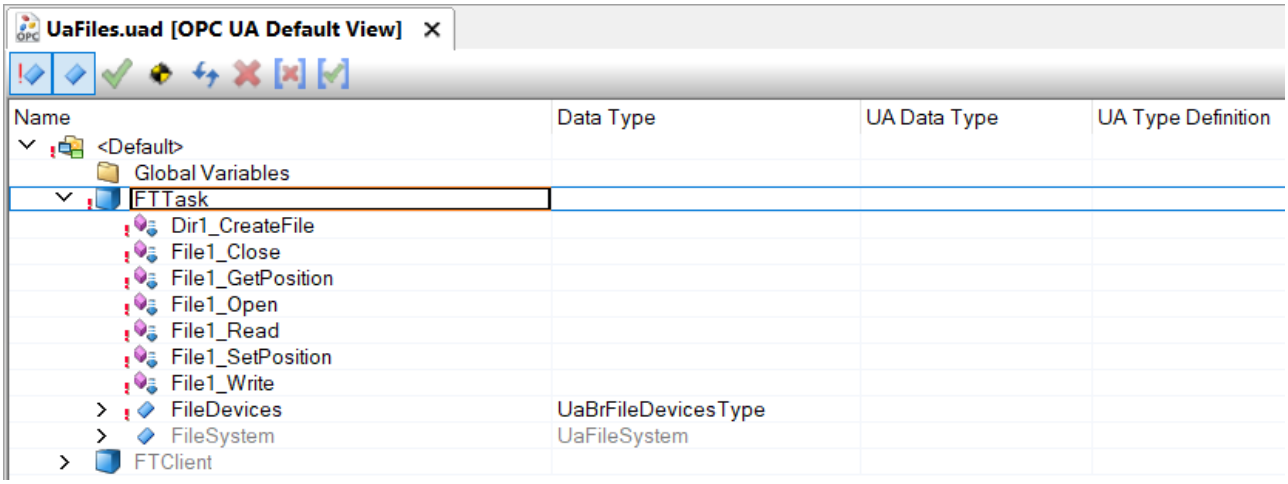
- Case 0..1:
  - UA_Connect to 'opc.tcp://localhost:4840'
- Case 2..3:
  - Get the NamespaceIndex of NamespaceUri := 'http://br-automation.com/OpcUa/FileTransfer/'

In CASE OpcReadfile.Step it does

- Case 0..2:
  - Wait for the Connect and get a Method Handle for the Open() method
- Case 3..4:
  - Call the Open() method with Mode 3 for the file ‘page1.jpg’
  - Store the FileHandle
- Case 5..6:
  - Get a Method Handle for the Read() method
- Case 7..8:
  - Call the Read() method for the FileHandle for only 10 Bytes of the file
  - Store the read data in PLC variable OpcReadfile.Data
- Case 9..10:
  - Get a Method Handle for the Close() method
- Case 11..12:
  - Call the Close() method for the FileHandle

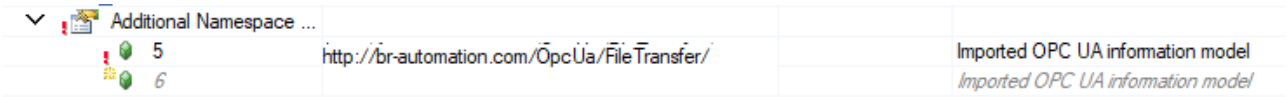
6 Default View

In the Default View, the variable and file transfer methods of the B&R information model are enabled and mapped with inverse references to the information model of the MassStorage objects.



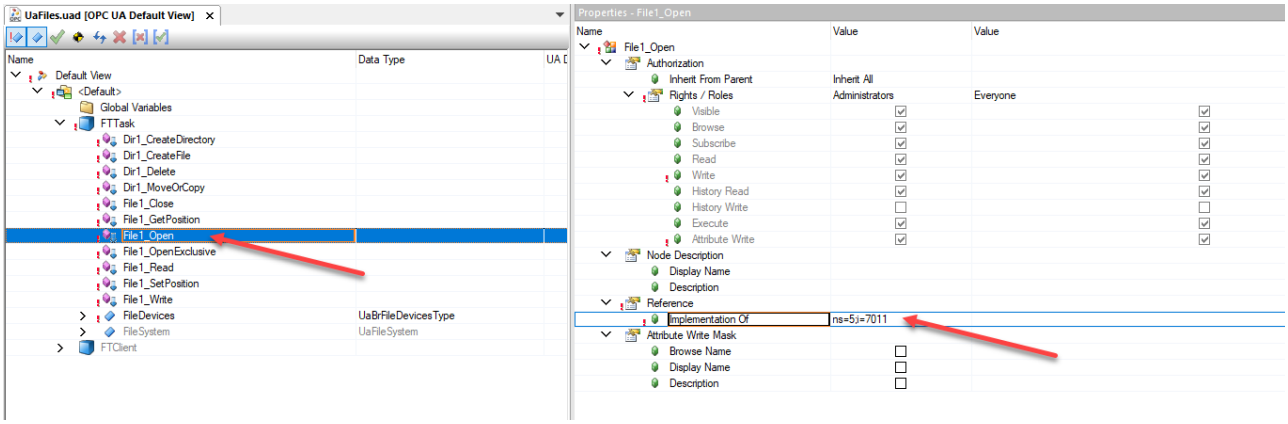
6.1 Namespace URI

The namespace URI is entered in the UaDvConfig.uadcfg. In this example, the namespace ns=5.



6.2 ImplementationOf references

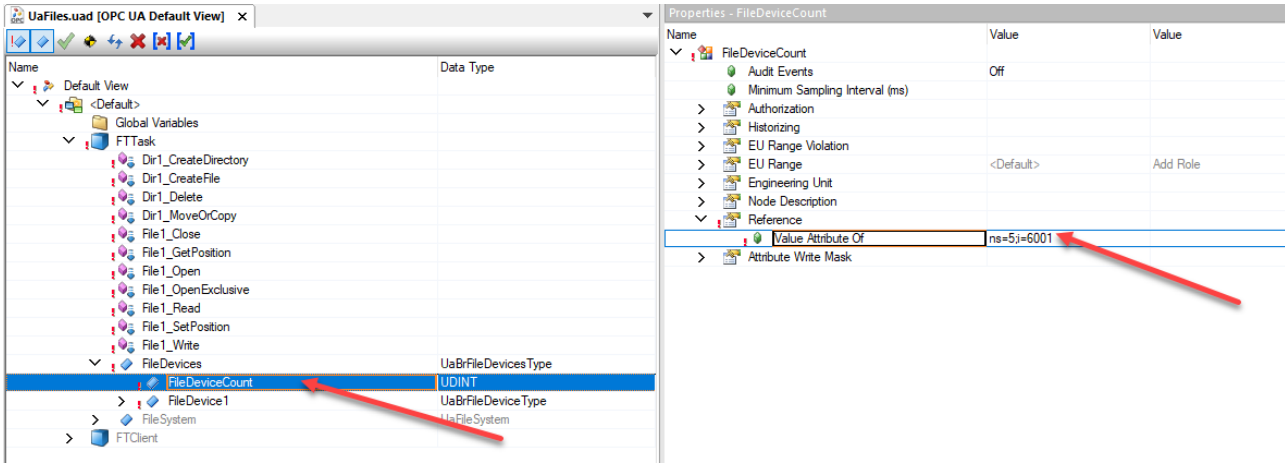
The inverse references (ImplementationOf) to the corresponding objects in the FileTransfer information model are then entered for the methods.
The namespace specified in the Default View (see 6.1) is used. In this example for the Open method ns=5;i=7011.



The NodeIds are determined either offline from the modeler tool or online with browsing.

6.3 ValueAttributeOf references

The inverse references (ValueAttributeOf) to the corresponding objects in the FileTransfer information model are then entered for the variables.
The namespace specified in the Default View (see 6.1) is used. In this example for the FileDeviceCount property ns=5;i=6001.



The NodeIds are determined either offline from the modeler tool or online with browsing.